

Informe Laboratorio 1 Verificación de Sistemas Digitales

Santiago Cadena - scadena@unal.edu.co

Carlos Rubio - cyrubio@unal.edu.co

Julio 28 de 2025

1. Introducción

El propósito de esta práctica es implementar el algoritmo de Euclides para calcular el Máximo Común Divisor (GCD) entre dos números, utilizando dos enfoques: una implementación en software y una simulación en hardware mediante SystemC. Posteriormente, se comparan los resultados de ambas versiones mediante el análisis de las formas de onda generadas.

2. Objetivos

- Introducir el uso de SystemC como herramienta de modelado a nivel de sistema.
- Comparar los enfoques software y hardware para la misma funcionalidad.
- Implementar una arquitectura en SystemC basada en módulos funcionales (ALU y Control).
- Validar la equivalencia funcional mediante simulación y visualización con GTK-Wave.

3. Fundamento Teórico

El algoritmo de Euclides permite calcular el GCD de dos números enteros positivos de forma iterativa, realizando restas sucesivas del número mayor con el menor

hasta que ambos sean iguales. Esta operación se puede representar mediante una máquina de estados finitos (FSM) y una unidad lógica-aritmética (ALU) que ejecuta la operación de resta.

4. Desarrollo

4.1. Configuración del Entorno

Se instaló SystemC 2.3.3 y la herramienta GTKWave para visualizar las formas de onda. Se creó un entorno de trabajo en `~/gcd_systemc2`.

4.2. Implementación en Software

También seguí la recomendación de utilizar herramienta ARCHC (disponible en <https://archc.sourceforge.net/index.html>).

Se instala por la Terminal de Linux utilizando los siguientes comandos.

```
sudo apt update
```

```
sudo apt install -y build-essential gcc-multilib flex bison
```

El archivo `gcd_software.cpp` contiene un módulo que calcula el GCD de forma iterativa. Utiliza un banco de pruebas que alimenta al módulo con los valores 48 y 18. El resultado esperado es 6.

Listing 1: Fragmento de `gcd_software.cpp`

```
while (x_reg != y_reg) {
    if (x_reg > y_reg) {
        x_reg = x_reg - y_reg;
    } else {
        y_reg = y_reg - x_reg;
    }
}
result.write(x_reg);
done.write(true);
```

Se genera el archivo `gcd_software.vcd` para validación.

4.3. Implementación en Hardware

La implementación hardware está dividida en tres módulos:

- **gcd_alu.cpp:** Implementa la ALU que ejecuta la resta dependiendo de la señal `sel`.
- **gcd_control.cpp:** Máquina de estados finitos que controla el proceso (IDLE, CHECK, SUBTRACT, DONE).
- **gcd_hardware.cpp:** Integra ALU y control, gestiona los registros internos y genera las señales de salida.

4.4. Banco de Pruebas

Ambas implementaciones cuentan con un banco de pruebas (Testbench) que aplica los valores de entrada y genera los archivos `.vcd` para visualización.

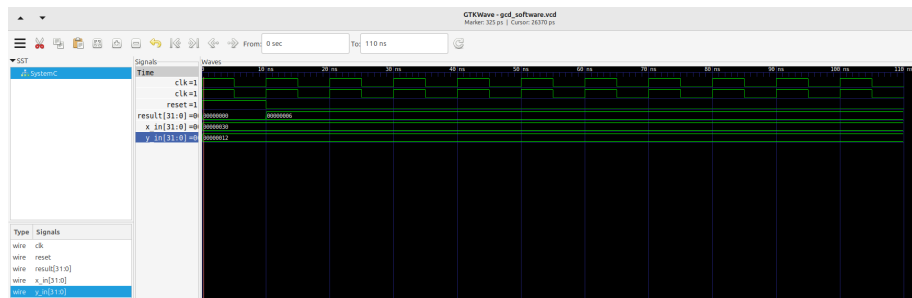


Figura 1: Forma de onda de la implementación software

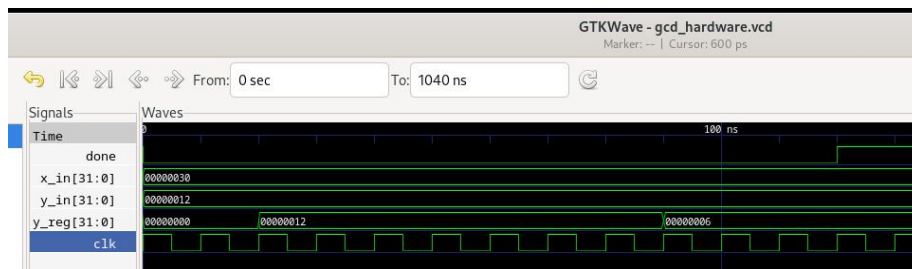


Figura 2: Forma de onda de la implementación hardware

XX

4.5. Makefile

Se utilizó un Makefile para automatizar la compilación, ejecución y apertura de las formas de onda:

- `make run` ejecuta ambas simulaciones.
- `make wave` lanza GTKWave con los archivos VCD.

5. Resultados

Ambas implementaciones convergieron correctamente al valor esperado: $\text{GCD} = 6$. Las formas de onda permiten verificar que el comportamiento del hardware replica adecuadamente la lógica funcional del software, respetando la secuencia: $48, 18 \rightarrow 30, 18 \rightarrow 12, 18 \rightarrow 12, 6 \rightarrow 6, 6$.

6. Conclusión

Se logró con éxito modelar y simular el algoritmo de Euclides en software y hardware, evidenciando las diferencias estructurales pero funcionalmente equivalentes entre ambos modelos. SystemC se mostró como una herramienta poderosa para la descripción de arquitecturas digitales. El uso de GTKWaves fue fundamental para la validación del comportamiento temporal de cada diseño.

7. Archivos Entregables

- `gcd_software.cpp`
- `gcd_hardware.cpp`
- `gcd_alu.cpp`
- `gcd_control.cpp`
- `Makefile`
- `gcd_software.vcd`
- `gcd_hardware.vcd`

- `simple_bus_main.cpp`
- `simple_bus_master_blocking.cpp`
- `simple_bus_tools.cpp`